

Query design

COMPLETE LEARNING GUIDE

What You'll Learn

Writing SQL that returns correct results is the easy half of query design; writing SQL that returns correct results efficiently, at production data volumes, is the part that separates a query that works in a demo from one that takes a service down under real load. This guide covers filtering and joins, aggregation semantics (the WHERE vs. HAVING distinction that trips up most learners), the choice between subqueries and CTEs for readability, the notorious N+1 query problem that ORMs make easy to accidentally introduce, using EXPLAIN to understand what the database is actually doing with your query, and parameterized queries as the non-negotiable defense against SQL injection.

1. SELECT Fundamentals and Filtering

A SELECT statement's clauses execute in a specific logical order that doesn't match the order you write them in: FROM/JOIN first, then WHERE (filtering individual rows), then GROUP BY, then HAVING (filtering grouped results), then SELECT (choosing/computing output columns), then ORDER BY, then LIMIT. Understanding this order explains behavior that otherwise seems confusing — for instance, why you can't reference a column alias defined in SELECT inside the same query's WHERE clause in most databases (WHERE is evaluated before SELECT computes that alias).

2. JOINS and Their Semantics

An INNER JOIN returns only rows that have a match in both tables — a student with zero enrollments would simply be excluded from a join between students and enrollments. A LEFT JOIN returns every row from the left table regardless of whether a match exists on the right, filling unmatched columns with NULL — the correct choice when you need 'all students, and their enrollments if any.' Choosing the wrong join type is a common source of quietly incorrect results: an INNER JOIN silently dropping rows you actually needed to see (with NULLs) is a very common real bug, not a hypothetical one.

SQL

```
-- All students and their enrollment count, including students with zero enrollments
SELECT s.id, s.name, COUNT(e.id) AS enrollment_count
FROM students s
LEFT JOIN enrollments e ON e.student_id = s.id
GROUP BY s.id, s.name
HAVING COUNT(e.id) > 0 -- only students with at least one enrollment
ORDER BY enrollment_count DESC;
```

3. Aggregation: GROUP BY and HAVING

GROUP BY collapses rows sharing the same value(s) in specified columns into a single row per group, at which point aggregate functions (COUNT, SUM, AVG, MAX, MIN) operate on each group rather than the whole table. WHERE filters individual rows before grouping happens; HAVING filters entire groups after aggregation, which is why 'only show courses with more than 10 enrolled students' requires HAVING COUNT(*) > 10 — WHERE cannot reference an aggregate result because it runs before aggregation exists.

4. Subqueries vs. CTEs

A subquery nests one query inside another, either in the WHERE clause (to filter using a computed set of values) or in the FROM clause (treating a query's result as a temporary table). A Common Table Expression (WITH clause) achieves the same thing but names the intermediate result and can be referenced multiple times in the outer query, and — critically for readability — lets you build a query as a sequence of clearly named logical steps instead of deeply nested parentheses. For queries with more than one or two levels of nesting, CTEs are almost always more maintainable.

SQL

```
WITH active_students AS (
    SELECT id, name FROM students WHERE status = 'active'
),
recent_enrollments AS (
    SELECT student_id, COUNT(*) AS cnt
    FROM enrollments
    WHERE enrolled_at > NOW() - INTERVAL '30 days'
    GROUP BY student_id
)
SELECT a.name, COALESCE(r.cnt, 0) AS enrollments_last_30d
FROM active_students a
LEFT JOIN recent_enrollments r ON r.student_id = a.id;
```

5. The N+1 Query Problem

The N+1 problem occurs when code fetches a list of N records with one query, then loops over them and issues a separate query per record to fetch related data — resulting in 1 + N total queries instead of 2. This is extremely easy to introduce accidentally through an ORM's lazy-loaded relations (accessing student.enrollments inside a loop over 500 students triggers 500 separate queries). The fix is eager loading — a single JOIN or a single batched IN (...) query that fetches all the related data for the entire list up front, which every

mainstream ORM supports explicitly (it's just not the default).

JAVASCRIPT

```
// N+1: one query per student inside the loop
const students = await db.student.findMany();
for (const s of students) {
  s.enrollments = await db.enrollment.findMany({ where: { studentId: s.id } });
}

// Fixed: a single query with eager loading
const studentsWithEnrollments = await db.student.findMany({
  include: { enrollments: true },
});
```

6. Reading Query Plans with EXPLAIN

EXPLAIN (and EXPLAIN ANALYZE, which actually runs the query and reports real timing) shows the execution plan the database chose: whether it used an index or scanned the whole table (a 'sequential scan' on a large table is usually the first red flag), the estimated versus actual number of rows at each step, and where the majority of time is being spent. Making a habit of running EXPLAIN on any query touching a large table before shipping it — rather than only after a production slowdown is reported — catches most performance problems while they're still cheap to fix.

7. Parameterized Queries and SQL Injection

Building a SQL string by concatenating raw user input directly into it is the source of SQL injection, one of the longest-standing and most damaging classes of web vulnerabilities — a crafted input value can terminate the intended query and append arbitrary additional SQL. Parameterized queries (placeholders like \$1 or ? that the database driver binds values to separately from the query structure) close this off entirely, because user input is never treated as executable SQL syntax, only ever as a literal value. Every mainstream database driver and ORM supports parameterization, and there is essentially never a legitimate reason to string-concatenate user input into a raw query.

JAVASCRIPT

```
// VULNERABLE: user input concatenated directly into SQL
const bad = `SELECT * FROM students WHERE email = '${userInput}'`;

// SAFE: parameterized – userInput is always treated as a value, never as SQL
const safe = await db.query("SELECT * FROM students WHERE email = $1",
[userInput]);
```

Common Pitfalls

- Introducing N+1 queries by accessing an ORM's lazy-loaded relation inside a loop instead of eager-loading it up front.

- Using `SELECT *` in production code, pulling unnecessary columns across the network and making the query more fragile to schema changes.
- Confusing `WHERE` and `HAVING` — trying to filter on an aggregate value in `WHERE`, which runs before aggregation exists.
- Choosing `INNER JOIN` when a `LEFT JOIN` was needed, silently dropping rows that had no match instead of showing them with `NULLs`.
- Building SQL queries via string concatenation of user input instead of using parameterized queries, opening a SQL injection vulnerability.
- Shipping a query against a large table without ever running `EXPLAIN` to check whether it's actually using an index.
- Writing a correlated subquery that re-executes once per outer row when a `JOIN` or a single aggregated subquery would compute the same result far more efficiently.

Key Takeaways

- SQL's logical execution order (`FROM`/`JOIN`, `WHERE`, `GROUP BY`, `HAVING`, `SELECT`, `ORDER BY`, `LIMIT`) explains most 'why can't I do this' query confusions.
- `WHERE` filters rows before grouping; `HAVING` filters groups after aggregation — they are not interchangeable.
- The N+1 problem is an ORM-era default trap; eager loading (a single batched query) is the fix, and it's opt-in in every mainstream ORM.
- `EXPLAIN`/`EXPLAIN ANALYZE` should be a routine check on any query touching a large table, not just a debugging tool used after an incident.
- Parameterized queries are the non-negotiable defense against SQL injection — never build queries via string concatenation of user input.
- CTEs (`WITH` clauses) generally beat deeply nested subqueries for readability once a query has more than one logical step.

Learn More

PostgreSQL Docs — The `EXPLAIN` Command

<https://www.postgresql.org/docs/current/using-explain.html>

MDN — Using Prepared Statements (SQL Injection Prevention)

https://developer.mozilla.org/en-US/docs/Web/Security/Attacks/SQL_Injection

PostgreSQL Docs — `WITH` Queries (CTEs)

<https://www.postgresql.org/docs/current/queries-with.html>